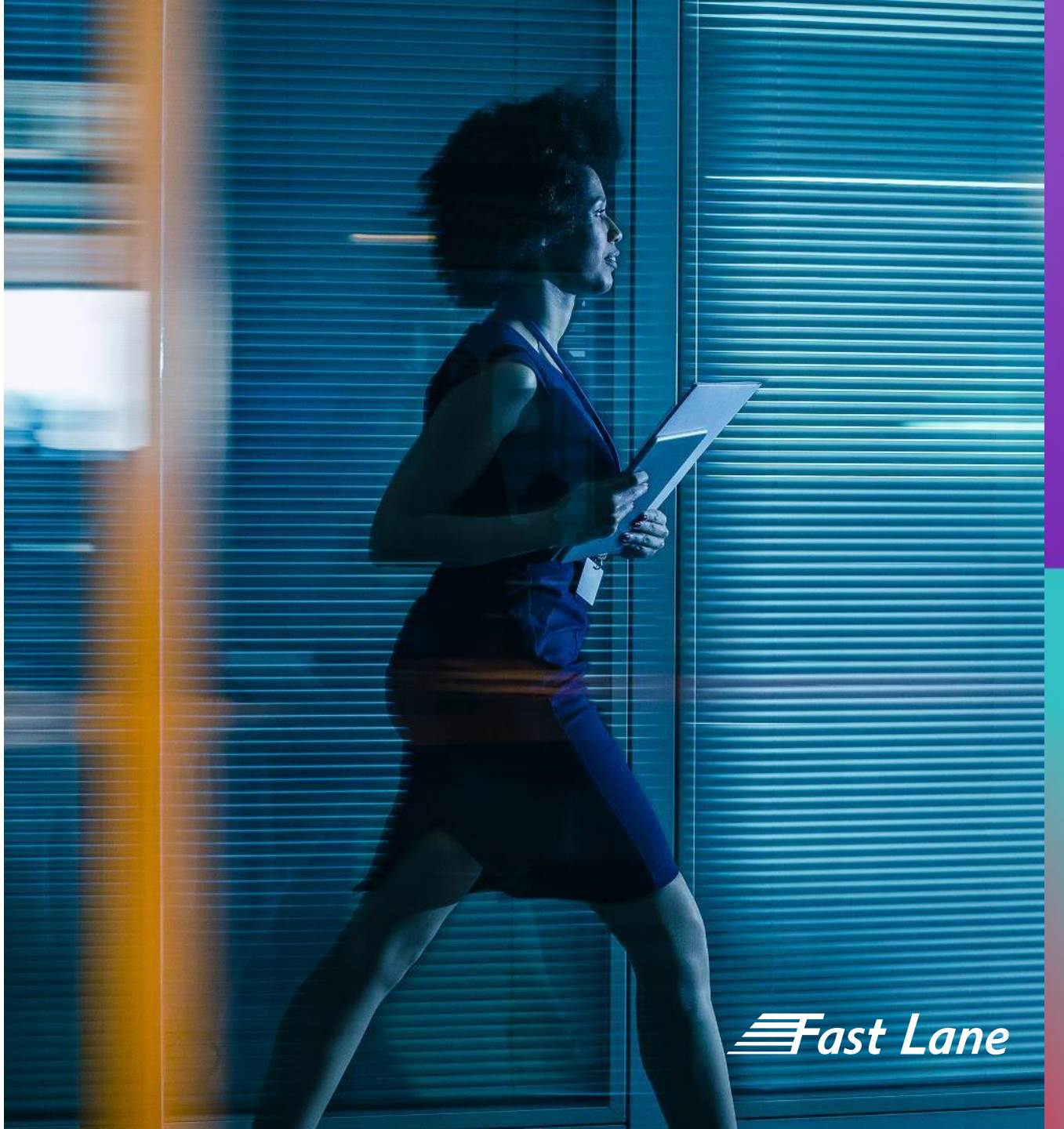
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 6

Event Sourcing & CQRS with Cosmos DB

Contoso Logistics keeps every shipment change as an event

Event-Driven Distributed Apps on Azure: Aspire, Dapr & Durable Execution



01

Transactional Outbox: Atomic Publishing

One batch for the event and its outbox, relayed to Service Bus

02

Change Feed Processor: Projections

Builder, leases, lag and change feed modes

03

CQRS: Separate Writes from Reads

Command and query handlers over separate containers

04

Read Models: Indexing, RUs, Consistency

Indexing policy, request charge and session tokens

05

Managed Redis: Change-Feed Cache Invalidation

Cache-aside, cleared by the projection after each scan

06

Event Store: Aggregates & Snapshots

Version-checked appends, replay and snapshots

07

Event Versioning: Evolve Schemas Safely

Additive changes, upcasting and CloudEvents

Event sourcing and CQRS with Cosmos DB

Contoso Logistics keeps every shipment change as an event

- **Atomic publishing**: the booking and its outbox event in one batch, relayed to Service Bus
- **Read models**: change feed projections, tuned queries and a Redis cache
- **Event store**: version-checked appends, replay, snapshots and safe schema changes



Overview

The path through event sourcing and CQRS

- Save the booking and its outbox event in one batch, relay it with the **change feed**
- Project the events into the **tracking read model** with a change feed processor
- Split commands from queries, then tune indexing, RUs and session tokens
- Serve the tracking page from **Azure Managed Redis**, cleared after each scan
- Store events per shipment, replay with snapshots and **version** the schemas



01

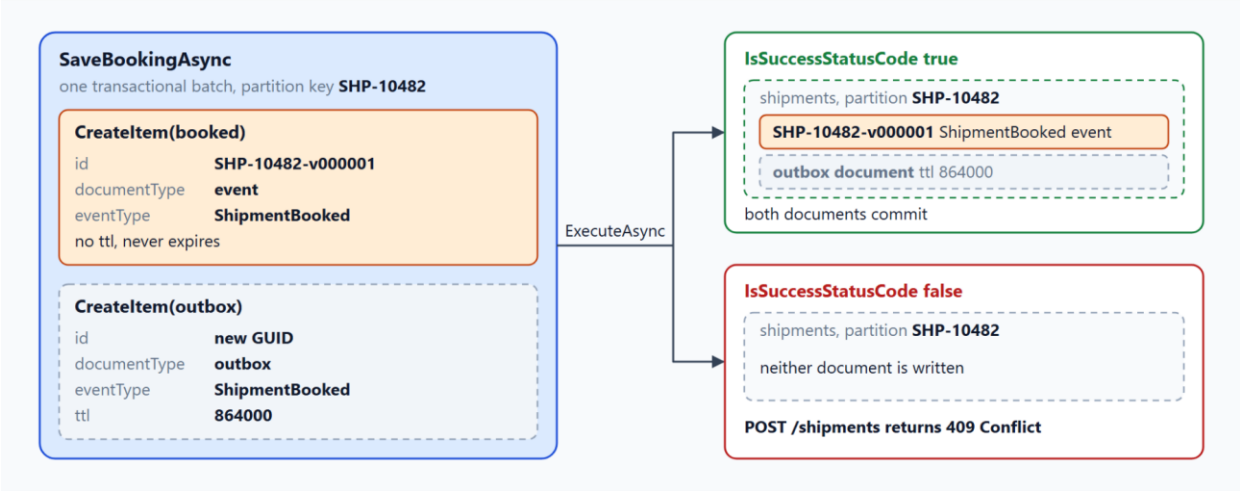
Transactional Outbox: Atomic Publishing

One batch for the event and its outbox, relayed to Service Bus

Transactional batch for the booking

The event and its outbox document commit together or not at all

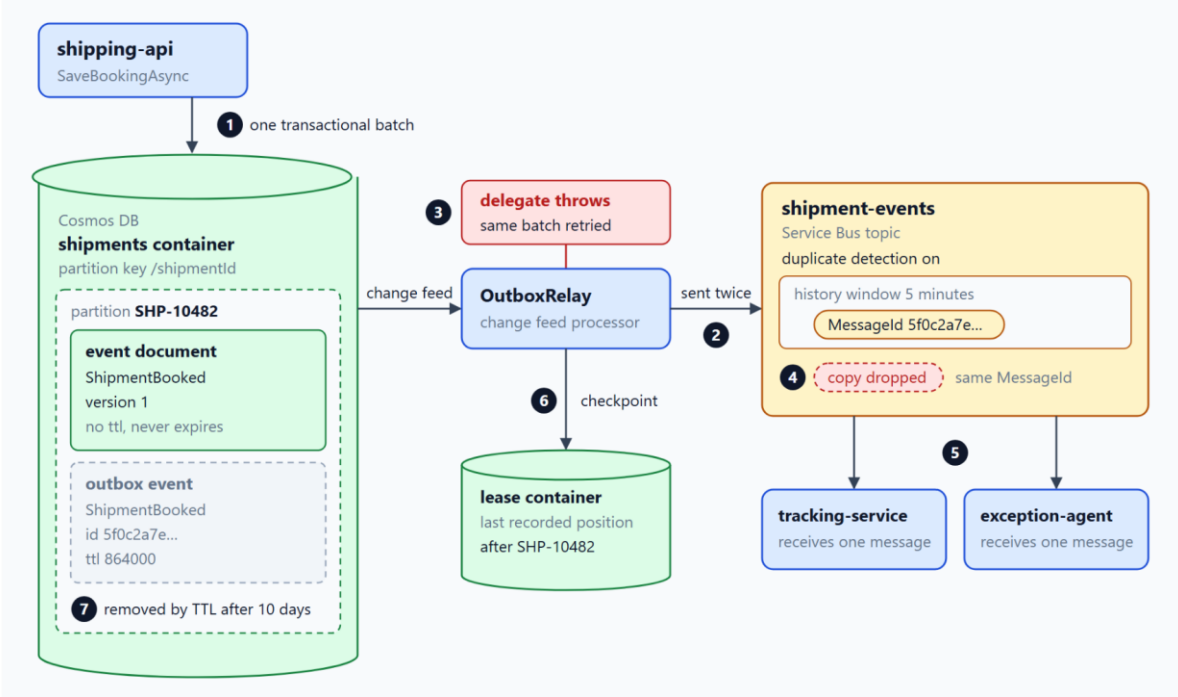
- A **dual write** to Cosmos DB and Service Bus can lose the event or announce a missing shipment
- The ShipmentBooked event and its outbox event share the partition key `/shipmentId`
- One **transactional batch**: one logical partition, up to 100 operations, at most 2 MB
- Check `IsSuccessStatusCode` before the caller hears success
- The outbox event carries `ttl` 864000 (ten days), the event document never expires



Relaying outbox events with the change feed

MessageId and duplicate detection drop the copies a retry sends

- A change feed processor reads new outbox documents and **checkpoints** in a lease container
- `MessageId` is the document `id`, `SessionId` is the shipment ID
- Every send is awaited; a throw retries the same batch, so some events go out twice
- **Duplicate detection** on `shipment-events` drops the copies (lab window: five minutes)
- TTL removes the outbox event after ten days, the event document stays



02

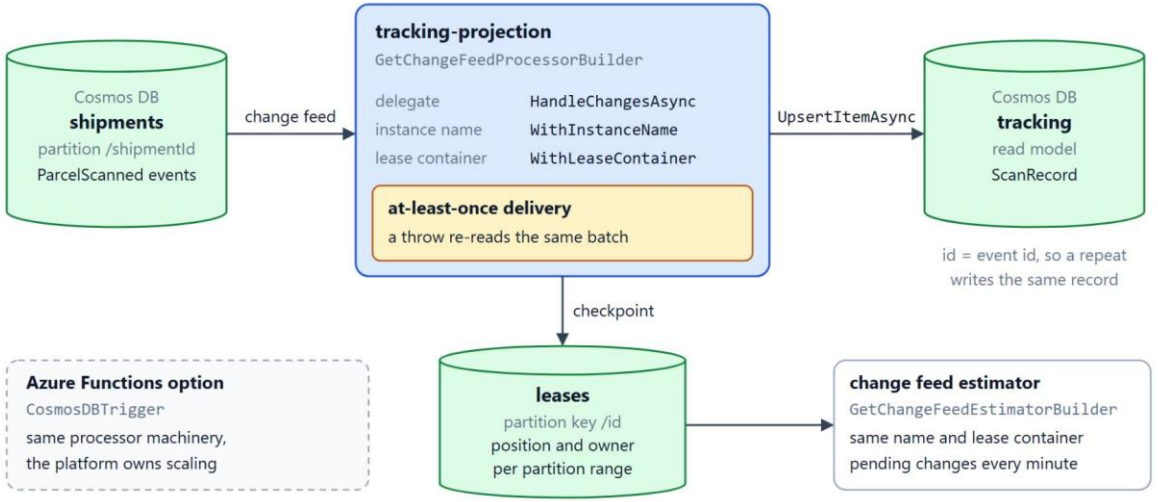
Change Feed Processor: Projections

Builder, leases, lag and change feed modes

Building the tracking projection

Processor name, delegate, lease container and the estimator

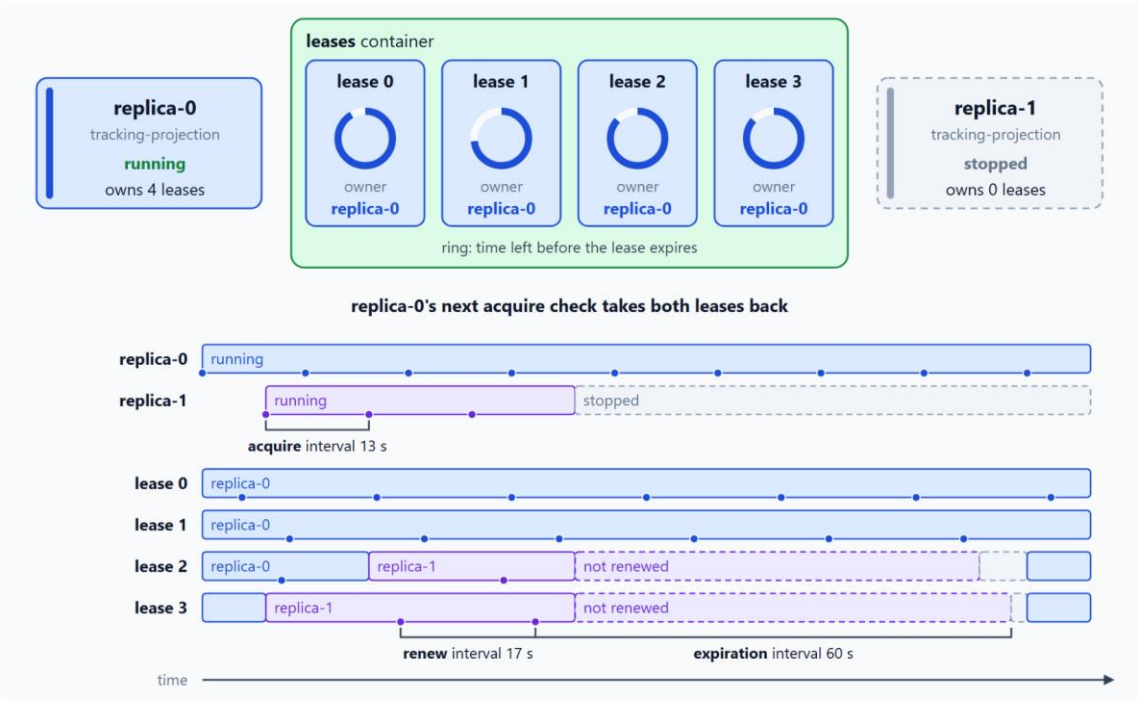
- `GetChangeFeedProcessorBuilder` takes a processor name and a delegate
- The **lease container** (partition key `/id`) stores each range's position and owner
- Delivery is **at least once**: a delegate that throws reads the same batch again
- The event ID is the ScanRecord ID, so a repeated upsert writes the same record
- The **change feed estimator** reports pending changes; `CosmosDBTrigger` is the Functions option



Leases and instances

Stable instance names and three intervals decide how work moves

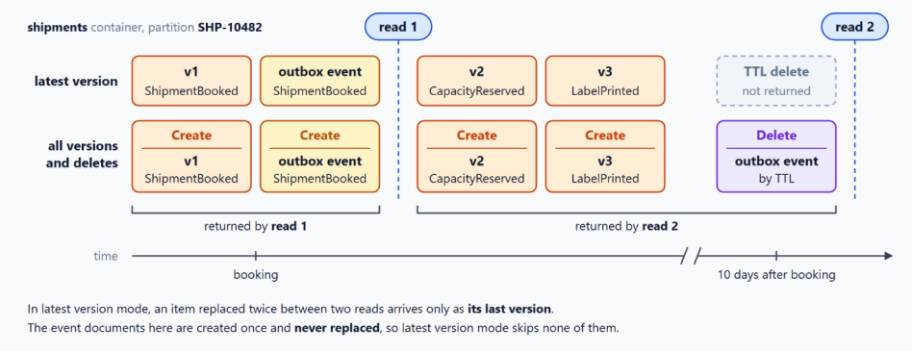
- Each running copy of the service is one **instance** of the processor
- A lease belongs to one instance at a time, and leases rebalance as instances join or stop
- Give every replica a stable `WithInstanceName` instead of a new GUID per start
- Acquire 13 s, renew 17 s, expiration 60 s: the Azure Functions trigger defaults



Latest version or all versions and deletes

What each change feed mode returns from the shipments container

- **Latest version** is the default and returns the newest state of each changed item
- It has no fixed retention and can start from the beginning, but it misses deletes
- **All versions and deletes** records every create, replace and delete, TTL included
- It needs continuous backup and starts only from now or a checkpoint
- The shipment audit needs deletes, so it uses `GetChangeFeedProcessorBuilderWithAllVersionsAndDeletes`



03

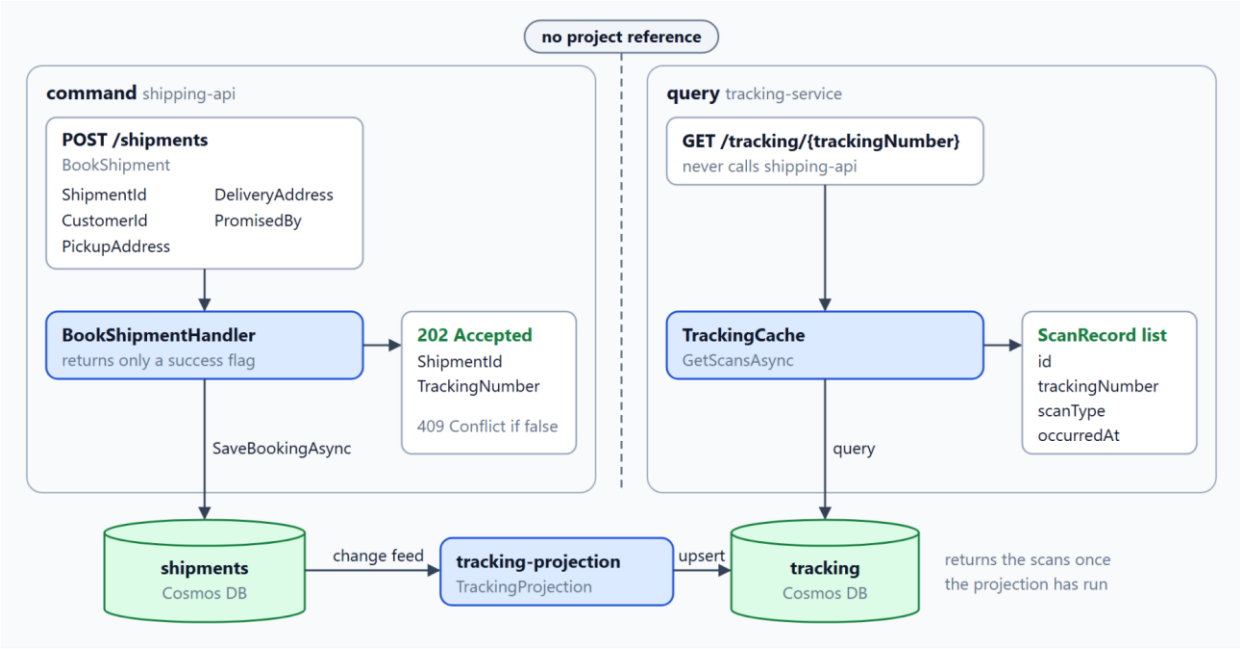
CQRS: Separate Writes from Reads

Command and query handlers over separate containers

Command side and query side

shipping-api writes the aggregate, tracking-service reads its own model

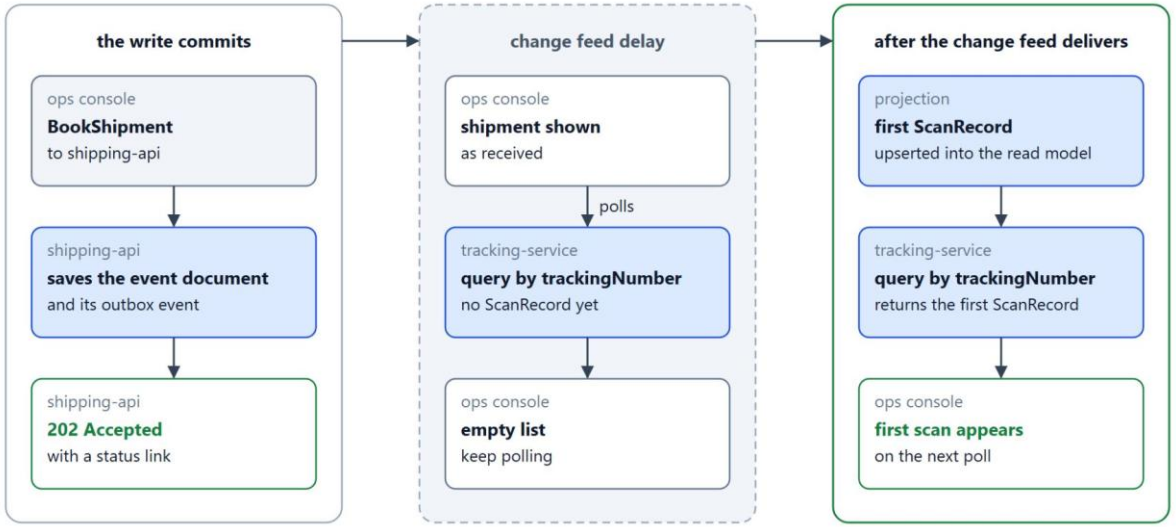
- A **command** names a business task such as `BookShipment` and returns only a success flag
- A **query** never changes state and returns a view shaped for one screen
- The event store partitions by `/shipmentId`, the read model by `/trackingNumber`
- The read model can be rebuilt from the event store at any time
- Plain handler classes registered in DI, no mediator library



Eventual consistency in the ops console

202 Accepted first, the first scan once the projection has run

- The projection runs after the write commits, so the read model can lag
- The booking endpoint answers **202 Accepted** with a status link
- The console shows the shipment as received and polls the tracking query
- An empty list means keep polling until the first scan appears
- **Session consistency** can close the gap for the same user



04

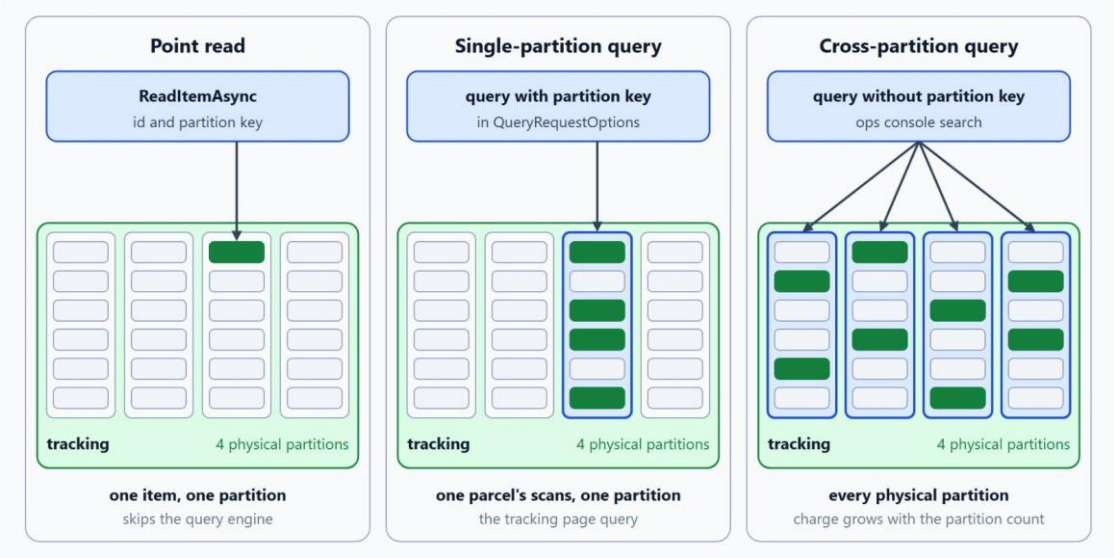
Read Models: Indexing, RUs, Consistency

Indexing policy, request charge and session tokens

Point reads and queries

Indexing and request charge on the tracking container

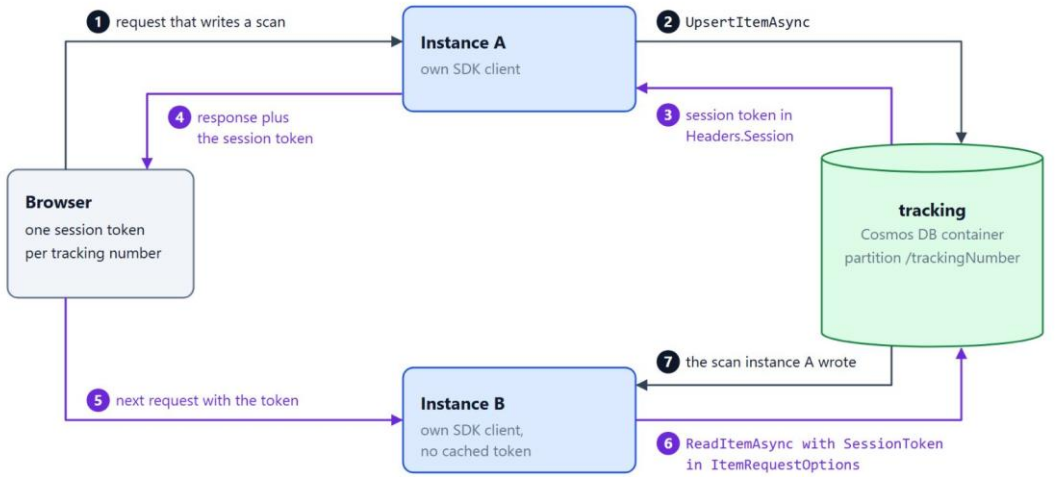
- Opt-out indexing: include `/*`, exclude `/_etag/?`
- A **composite index** on `scanType` ascending and `occurredAt` descending matches the ops console sort
- Log `RequestCharge` and `ActivityId` on every read you tune
- A **point read** by `id` and partition key skips the query engine and costs the least
- The partition key in `QueryRequestOptions` keeps the page query on one partition



Session tokens for read-your-writes

The token travels with the next request to another instance

- **Session** is the default consistency level: a client reads its own writes
- Each write returns a session token in `Headers.Session`
- Several web instances don't share the SDK's cached token
- The browser gets the token and sends it back with the next request
- Tokens are partition bound, so keep one per tracking number



05

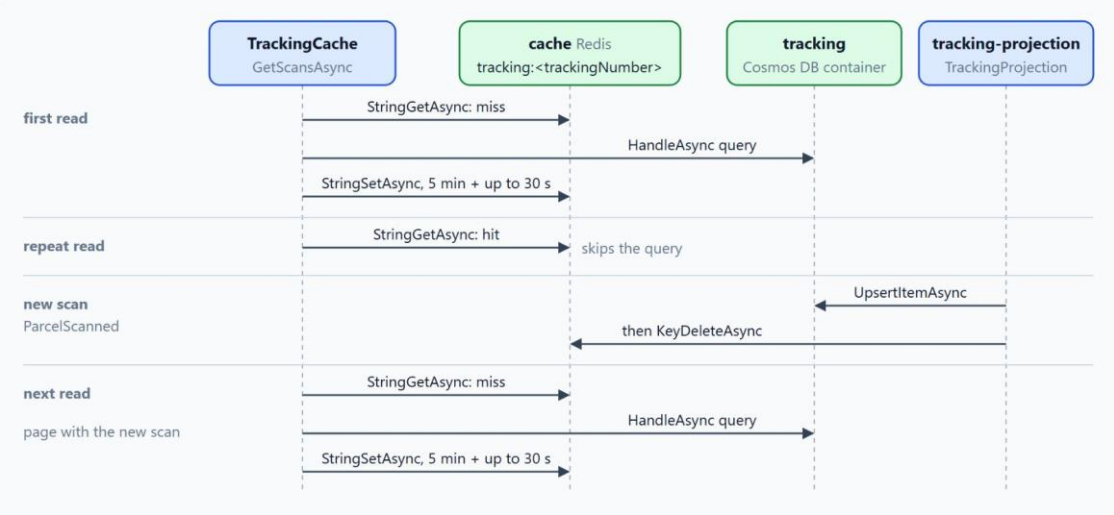
Managed Redis: Change-Feed Cache Invalidation

Cache-aside, cleared by the projection after each scan

Cache-aside with change feed invalidation

The projection deletes the cached tracking page after each scan

- **Cache-aside**: read `tracking:<trackingNumber>`, query on a miss, store with an expiration
- Five minutes plus up to 30 seconds of **jitter** spreads the expirations
- The projection upserts the scan first, then calls `KeyDeleteAsync`
- A retry repeats the idempotent upsert, and the TTL bounds a stale page
- In Azure: Entra ID through `Microsoft.Azure.StackExchangeRedis`, wired by Aspire



06

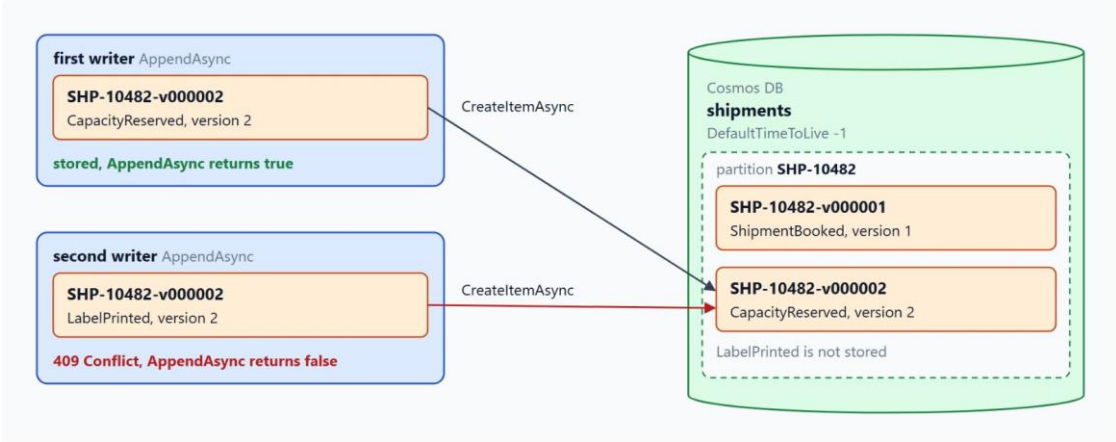
Event Store: Aggregates & Snapshots

Version-checked appends, replay and snapshots

Appending events with a version check

The event ID carries the version, so a second append gets 409 Conflict

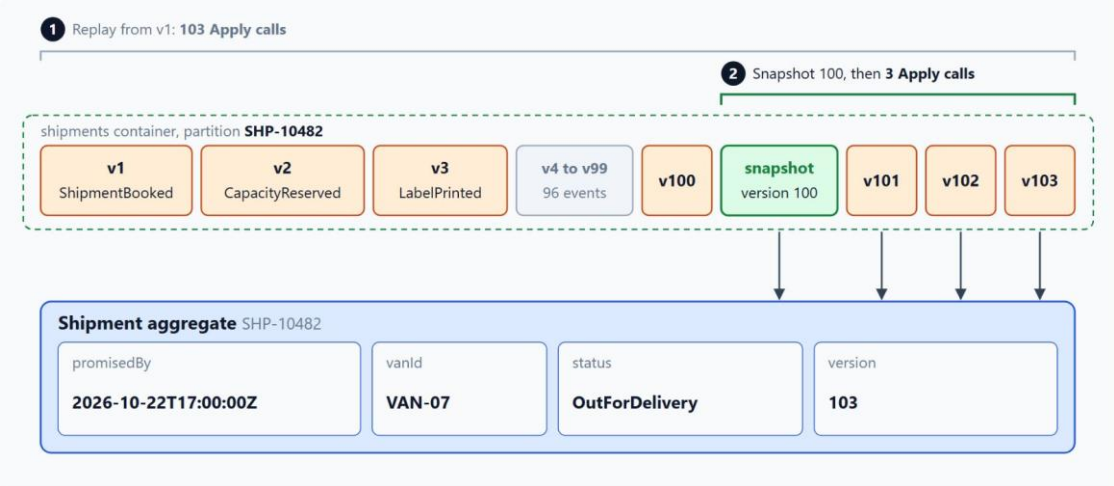
- One immutable event document per change, partition key `/shipmentId`, no `ttl`
- A correction is a new event: `PaymentRefunded` follows `PaymentCaptured`
- The ID `{shipmentId}-v{version:D6}` makes a second append for the same version collide
- `CreateItemAsync` answers **409 Conflict** and `AppendAsync` returns false
- `IfMatchEtag` protects documents that get replaced; append-only events don't need it



Replay and snapshots

Load from the newest snapshot and apply only the events after it

- `LoadAsync` queries one partition in version order and calls `Apply` per event
- The aggregate reads nothing but its own events
- A **snapshot** stores the state at one version, in the same partition
- 103 events: snapshot 100, then 3 `Apply` calls instead of 103
- The snapshot ID encodes its version, so a duplicate create gets a 409 to ignore



07

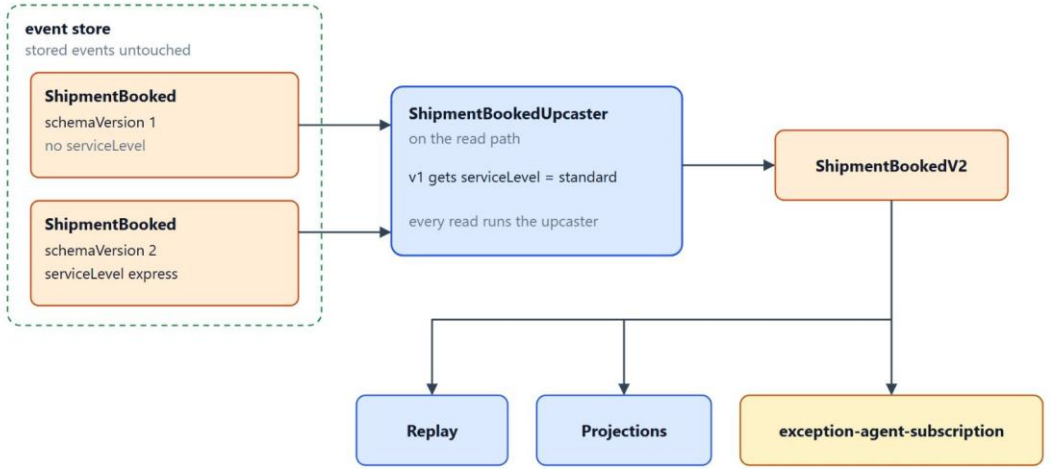
Event Versioning: Evolve Schemas Safely

Additive changes, upcasting and CloudEvents

Upcasting on read

Stored events stay untouched, readers see the newest shape

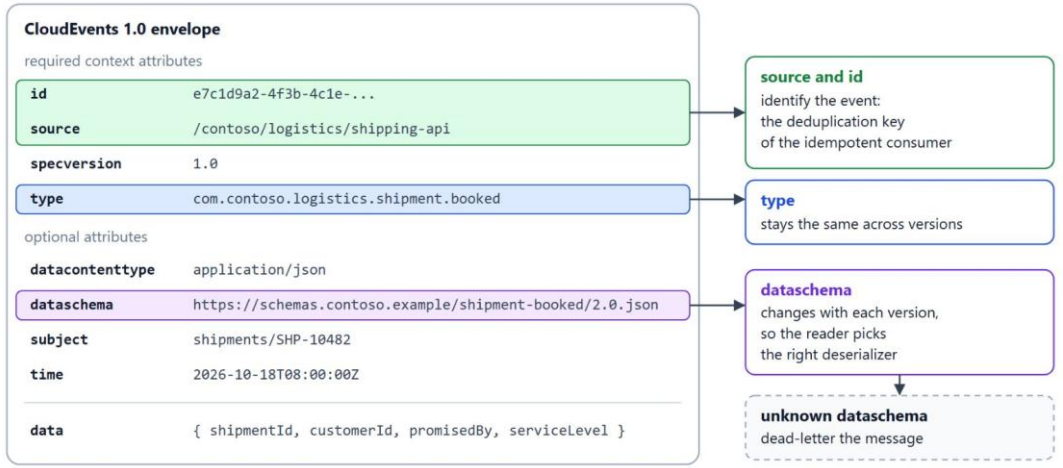
- Past events are **immutable**: correct them with a compensating event or an upcaster
- An **additive** change adds an optional field, and tolerant readers ignore unknown fields
- A missing `serviceLevel` deserializes as null, and the reader applies `standard`
- `ShipmentBookedUpcaster` turns `schemaVersion` 1 into `ShipmentBookedV2` on read
- Upcasters chain one version per step, and every read runs them



CloudEvents type and dataschema

Consumer rules for each kind of schema change

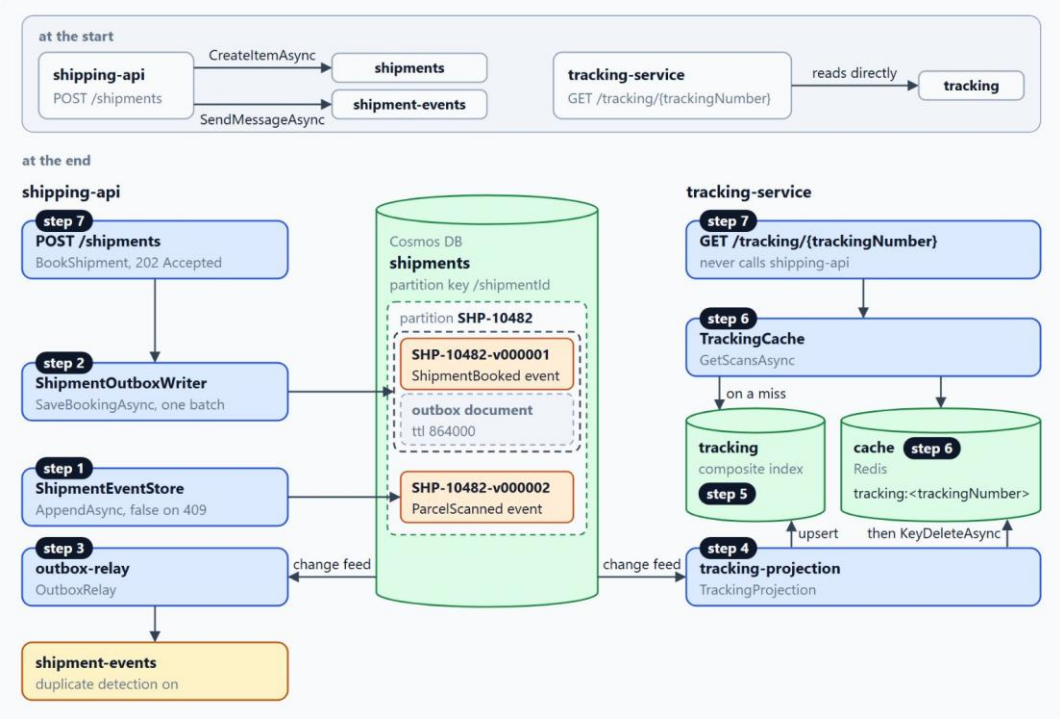
- `type` stays the same across versions, `dataschema` changes with each one
- `source` plus `id` is the idempotent consumer's deduplication key
- Adding an optional field is safe for consumers that already run
- A rename writes both names for one release; a new meaning gets a new `type` or `dataschema`
- A consumer dead-letters a message whose `dataschema` it doesn't know



Lab: event-source shipments with an outbox

From two separate calls to an event store, an outbox and a cached read model

- Steps 1 and 2: an event store that rejects a second event per version, booking and outbox in one batch
- Step 3: `outbox-relay` publishes the outbox documents to `shipment-events`
- Steps 4 and 5: `tracking-projection` fills the indexed `tracking` container
- Step 6: Redis caches the tracking page and drops it after each scan
- Step 7: `BookShipment` answers 202 Accepted, `GET /tracking/{trackingNumber}` reads the model



Takeaways

What to carry into the Dapr module

- Commit the event and its outbox event in one batch, relay with duplicate detection
- Make every projection idempotent: upsert under the event ID, await every write
- Keep reads on one partition, and send session tokens back for read-your-writes
- Delete the cached page after the upsert, and let the TTL bound the rest
- Put the version in the event ID, and evolve schemas additively or by upcasting

